🧭 Nuance Protocol — Master Build Plan

> **Production Hardening, Intelligent Contract Security, Fund Rescue Architecture & Release Roadmap**
> *Target Network:* GenLayer Testnet Bradbury (Chain ID 4221) → Mainnet
> *Version:* 1.0 — Production Readiness

---

1. Executive Summary & Root Cause Diagnosis

During live testing on GenLayer Testnet Bradbury, deposits and stakes in Intelligent Contracts succeeded, but withdrawals, payouts, and refunds failed to reach user wallets. Furthermore, deployers had no administrative rescue controls.

The Two Root Causes

1. **GenLayer Platform-Level Execution Bug (`genlayerlabs/genvm-manager#20`)**: `emit_transfer(value=..., on="finalized")` logs the transfer event into the transaction receipt, but validator execution nodes currently fail to dispatch native GEN to destination addresses.
2. **Smart Contract Architectural Gaps**:
3. Contracts (`NuanceEscrow` , `NuancePredictionMarket` , `NuanceGovernance`) lack `admin` roles and `emergency_refund()` functions.
4. Internal state (e.g. `self.funded_amount = 0` , `claimed = 1` , `stake = 0`) zeroes out immediately, preventing users from retrying failed payouts.
5. `NuanceDisputeCourt` and `NuanceEscrow` are unlinked: court rulings cannot be received or executed by the escrow contract, freezing disputed funds permanently.

---

2. Phase 1: Smart Contract Upgrades & Redeployment (Critical)

1.1 Emergency Withdrawal & Admin Escape Hatch

Add an `admin` role and an emergency refund function to all contracts holding value:

```
# contracts/nuance_escrow.py
@gl.public.write
def emergency_refund(self, recipient_override: str = "") -> None:
    if gl.message.sender_address != self.admin and gl.message.sender_address != self.
        raise gl.vm.UserError("Unauthorized: Only admin or creator can invoke emergen
```

```
        if self.funded_amount <= 0:
            raise gl.vm.UserError("No funded amount to refund.")

        refund_target = Address(recipient_override) if recipient_override else self.creat
        amount = self.funded_amount
        self.funded_amount = 0
        self.status = "emergency_refunded"

        recipient = gl.get_contract_at(refund_target)
        recipient.emit_transfer(value=u256(amount), on="finalized")
```

1.2 Connecting Dispute Court to Escrow

- Store `court_address: Address` in `NuanceEscrow.__init__`.
- Implement `execute_court_ruling(dispute_id: u256)`:
- Calls `NuanceDisputeCourt.get_dispute(dispute_id)`.
- Verifies `ruling == "favor_respondent"` (payout counterparty) or `ruling == "favor_claimant"` (refund creator).
- Unfreezes and transfers the milestone collateral accordingly.

1.3 Prediction Market Cancellation & Unresolved Market Safety

- Add `cancel_market(reason: str)` to `NuancePredictionMarket`:
- If the resolution URL 404s, times out, or the LLM cannot determine the outcome, allows bettors on both YES and NO to withdraw their original stakes.
- Fixes edge cases where 100% of bets were placed on the losing side by returning the pool.

1.4 Safe Payable Deposit Guard

- Prevent accidental loss of funds when wrong senders call payable functions like `fund_escrow`.

---

3. Phase 2: Native Transfer Settlement & Relayer Workaround

2.1 Protocol Settlement Verification

- Continue tracking both the contract's self-reported ledger (`funded_amount`) and the chain's actual native balance (`eth_getBalance` via `contract_balance`).
- Verify whether GenLayer requires a specific contract recipient interface (`@gl.public.write.payable` / `__receive__`).

2.2 Fallback Claim Relayer & Faucet Rebalancing

- **Automated Relayer Daemon**: Backend daemon that monitors `FINALIZED` receipts for `claim_winnings`, `cancel_escrow`, and `retract_vote`.

- If `contract_balance` remains unchanged 60 seconds post-finalization, the relayer automatically disburses testnet tokens directly from the deployer faucet to the user's wallet, ensuring no user is stranded.

---

4. Phase 3: Backend & Infrastructure Hardening

3.1 Direct Python JSON-RPC Client (Eliminating Subprocesses)

- Currently, `services/genlayer_indexer.py` spawns `npx tsx scripts/genlayer-read.ts` child processes continuously.
- **Refactor**: Replace subprocesses with an async Python `httpx` JSON-RPC client querying `https://rpc-bradbury.genlayer.com`.
- **Result**: Drops RAM consumption by 180MB and eliminates CPU spikes, preventing Render free/standard tier OOM crashes.

3.2 Database Engine Alignment

- Keep all datetime columns timezone-aware (`DateTime(timezone=True)`) across both SQLite and Render PostgreSQL.
- Maintain idempotent Alembic migrations.

---

5. Phase 4: Frontend & UX Polish

5.1 Environment Configuration Synchronization

Update `.env.local` and Vercel environment settings to declare all 6 contract addresses:

```
NEXT_PUBLIC_API_URL=https://nuance-backend-XXXX.onrender.com
NEXT_PUBLIC_ESCROW_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_DISPUTE_COURT_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_PREDICTION_MARKET_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_GOVERNANCE_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_VALIDATORS_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_AGENT_DIRECTORY_CONTRACT_ADDRESS=0x...
```

5.2 Full On-Chain Cutover & Wallet Signing Restoration (Escaping Demo Mode)

Diagnosis of the "Demo Mode" Behavior: Currently, Governance and Dispute Court do not prompt MetaMask to sign transactions because the frontend employs safety fallbacks: - **Governance**: Evaluates `const isOnChain = pr.onChainProposalId ≠ null`. If `onChainProposalId` is absent (due to stale Bradbury testnet addresses or unlinked database rows), it falls back to the off-chain mock vote button to avoid breaking the UI. - **Dispute Court**: Evaluates `const isOnChainFiled = disputeData.chain_status ≠ "legacy_offchain"`.

Unlinked disputes default to `legacy_offchain`, bypassing wallet signatures for evidence and escalation.

Action Plan to Activate Live Wallet Signing: 1. **Redeploy & Bind Contract Registries**: Deploy fresh `NuanceGovernance` and `NuanceDisputeCourt` contracts to Bradbury testnet and record their addresses in Vercel (`NEXT_PUBLIC_GOVERNANCE_CONTRACT_ADDRESS` , `NEXT_PUBLIC_DISPUTE_COURT_CONTRACT_ADDRESS`) and Render (`backend/.env`). 2. **On-Chain Seed & Indexer Synchronization**: - Run `create_proposal_on_chain` so each proposal receives a valid index on the `NuanceGovernance` contract (`on_chain_proposal_id ⩾ 0`). - Run `file_dispute_on_chain` for disputes so they obtain on-chain tracking IDs (`on_chain_dispute_id ⩾ 0`). 3. **Frontend Wallet Signing Triggers**: - **Governance**: Instantly unlocks "Stake GEN to vote" preset chips (0.5, 1, 2, 5 GEN) and prompts MetaMask via `castVoteOnChain()` (`@gl.public.write.payable`). - **Dispute Court**: Prompts MetaMask via `fileDisputeOnChain()` and `submitEvidenceOnChain()` against the real GenVM validator committee. - **Escrows**: Prompts MetaMask via `fundEscrowOnChain()` payable deposits. - **Prediction Markets**: Prompts MetaMask via `placeBetOnChain()` payable odds staking.

5.3 Actionable Resolution UX

- Replace static "stuck" notices in `escrow-detail-view.tsx` and `prediction-detail-view.tsx` with live status indicators and a one-click **"Request Faucet Relayer"** / **"Retry On-Chain Claim"** button.

---

6. Implementation Timeline & Milestones

Milestone	Deliverables	Est. Timeline	Status
M1: Smart Contract Patches	Emergency refund, dispute hook, market cancel	Days 1–2	Ready to patch
M2: Contract Redeployment	Deploy all 6 contracts via `scripts/deploy.ts` & update Vercel/Render envs	Day 2	Pending M1
M3: On-Chain Cutover	Bind Governance & Court to chain IDs; restore MetaMask signing flows	Days 2–3	Planned
M4: Native RPC Indexer	Replace subprocess calls with Python `httpx` RPC	Day 3	Planned
M5: Payout Relayer	Automatic faucet disbursement for stuck claims	Day 4	Planned

Milestone	Deliverables	Est. Timeline	Status
M6: Production Release	Live E2E tests, Vercel & Render production cutover	Day 5	Ready